



Programación de estructuras de datos y algoritmos fundamentales

(Gpo 850)

TC1031.850

Después de clase

Act-Integradora-1

Conceptos básicos y algoritmos fundamentales

Laura Cintora Cemdejas

A01712379

Profesor

Eduardo Arturo Rodríguez Tello

Fecha

25 de Marzo del 2025

Reflexion:

1. Analiza la importancia y eficiencia del uso de los diferentes algoritmos de ordenamiento y búsqueda en una situación problema de esta naturaleza.

La utilidad de algoritmos, son rápidos y eficientes en el área de ordenar y buscar porque para la gestión de mayores cantidades de datos, en este caso como la bitácora de registros y los intentos de ataque a una red. Por lo que ordenar los registros nis permite que las operaciones de las consultas sean más veloz y eficaz. Como consecuencia facilita la detención de patrones misteriosa, también ayuda a mejorar la capacidad de la respuesta ante cualquier amenaza de seguridad.

Para la parte de la organización de registros, se implementó mezclas de CountingSort y QuickSort. El primero nos permitió un orden de forma eficiente para los datos, el cual están con base a los días del año con la complejidad de $O(n+k)$, por lo tanto QuickSort hace que haya un mejor orden en la parte interna de los registros de $O(n \log n)$. Donde esta combinación se desarrolla el tiempo de ordenamiento a diferencia del uso único de algoritmos con base a comparaciones de Merge Sort o Heap Sort.

Durante la realización del algoritmo de ordenamiento, se hicieron medidas de comparaciones y swaps, sin embargo, dentro del programa completo la tarea en 73 milisegundos, entonces esto nos indica que tuvo su desempeño de forma correcta. Por lo que el ordenamiento QuickSort es el indicado para este problema. Así que 113,741 comparaciones son menor que $O(n^2)$ por lo que indica que el QuickSort funciona de forma correcta en su caso promedio $O(n \log n)$.

Mientras que 68,346 swaps nos indica que es la estrategia que divide y ordena en dos pasos, el cual nos ayuda a minimizar el número de intercambios innecesarios. Por lo que la combinación permitió agrupar datos similares antes de aplicar QuickSort, por lo que muestra que es un método efectivo para optimizar el rendimiento.

Counting Sort es eficiente para ordenar los datos en un rango pequeño, y el impacto que genera es el tamaño de los grupos que QuickSort necesita ordenar, disminuye los swaps y comparaciones.

Porque use este en lugar de otros algoritmos debido a que los otros métodos solo tiene el $O(n^2)$ y el otro $O(n \log n)$ por lo que el tiempo de ejecución es más eficaz incluso con valores de datos grandes. Cabe mencionar que se utilizó la búsqueda binaria en lugar de búsqueda lineal porque nos permite utilizar una búsqueda de la bitácora de forma veloz además el uso de punteros `find_by_interval` se utilizó en lugar de un doble for anidado, debido a que nos ayuda a encontrar pares de registros separados por D de forma eficaz. Así que cuando el administrador de la red necesita encontrar los intentos de ataques específicos se reducía de forma drástica el tiempo de búsqueda con el uso de “búsqueda binaria” y esto hace que sea viable el análisis de las bitácoras grandes sin que se afecte el rendimiento

En resumen, el counting custom = $O(n+k)$ al final son asignación directa y sus números de asignaciones en la última fase $O(n)$ y se puede reportar los swaps como la cantidad de asignación en la fase final del algoritmo.

Pseudo-código:

Funcion encontrar_par_con_diferencia_D(registros, D):

```

l = 0 // Índice izquierdo
r = 1 // Índice derecho
mientras (l < r y r < registros. Tamaño):
    diferencia = registros[r](DIA) - registros[l](DIA)
    si diferencia == D:
        retornar (l, r) // Se encontró el par de registros
    si no si diferencia > D:
        l = l + 1 // Mueve el índice izquierdo para reducir la diferencia
    inosi no:
        r = r + 1 // Mueve el índice derecho para aumentar la diferencia
retornar (-1, -1) // No se encontró un par con diferencia D

```

Este algoritmo se implementó para poder ubicar y encontrar el primer registro de D el cual está ubicado en el archivo `Search.h` y `Search.copp`. Por lo que la búsqueda de intervalo con punteros va recorriendo en la lista ordenada.

Conclusión:

Realizar este proyecto fue todo un reto, el cual enfrente muchas complicaciones, pero se logró, fueron desveladas, pero la perseverancia y la constancia logro sacar el trabajo, considero que no soy buena explicando a veces lo que hago el cual es un comentario que muchos profes me dicen, pero es algo que debo mejorar en comunicar y hablar de forma clara y sencilla lo que hago porque a veces me complico demasiado. Entonces, la comparación entre la complejidad teórica y los resultados empíricos nos demostró que el diseño que elegimos e implementamos fue la solución acertada, ya que se logró junto con mi compañero un tiempo de ejecución de forma consistente con el uso de algoritmos empleados. Acidez CountingSort y QuickSort garantiza un ordenamiento rápido, mientras que la búsqueda binaria permite consultar de forma rápida. Optamos por la técnica de dos punteros porque es una de las soluciones más eficientes para este problema. A diferencia de otros métodos tradicionales para buscar pares, esta estrategia evita un procesamiento excesivo y mejora significativamente el rendimiento.

Referencias bibliográficas:

Improve, K. F. (2014, enero 28). Binary Search Algorithm - iterative and recursive implementation. GeeksforGeeks.

https://www.geeksforgeeks.org/binary-search/?ref=gcse_outind

Improve, K. F. (2014a, enero 7). Quick sort. GeeksforGeeks.

<https://www.geeksforgeeks.org/quick-sort-algorithm/>

Improve, K. F. (2013, marzo 18). Counting Sort - data structures and algorithms tutorials.

GeeksforGeeks. <https://www.geeksforgeeks.org/counting-sort/>